

CHAPITRE X

PROCÉDURES STOCKÉES

I. INTRODUCTION

Cette unité traite des programmes et des **vues stockés**, qui sont des objets de base de données définis en termes de code **SQL** stocké sur le serveur pour un **appel ultérieur**.

Les procédures stockées elles-mêmes peuvent être écrites en utilisant **PL / SQL**. Des procédures stockées existent pour effectuer des opérations gourmandes en données qu'il est impossible d'accomplir à l'aide de **SQL** uniquement. Elles effectuent ces opérations dans la base de données lorsque les performances du réseau sont un problème superficiel.

OBJECTIFS

Après avoir étudié cette unité, vous devriez être capable de:

- ❖ Définir et distinguer entre les **procédures stockées** et les **routines**
- ❖ Expliquer avec des exemples les syntaxes de **Create Procedure** et **Create Function**
- ❖ Expliquer avec des exemples les syntaxes de **Alter Procedure** et **Alter Function**
- ❖ Décrire avec des exemples les syntaxes de **Drop Procedure** et **Drop Function**
- ❖ Expliquer l'importance de **Show Create Procedure** et **Show Create Function**
- ❖ Expliquer l'importance d'Afficher l'état de la procédure et de l'état de la fonction
- ❖ Décrire l'utilisation de **Call Statement**

II. PROCEDURES STOCKEES ET ROUTINES

En fait, une procédure stockée désigne collectivement des fonctions stockées, des procédures autonomes, des fonctions empaquetées et des procédures autonomes.

Pour appeler une procédure stockée, vous devez connaître **le nom de la procédure** et **connaître les paramètres qui lui seront transmis**. Dans le cas d'une **fonction**, vous devez également connaître **le type de retour**.

Les programmes stockés incluent les objets suivants:

- a) **Routines stockées**, c'est-à-dire fonctions et procédures stockées. Une fonction stockée est utilisée un peu comme une fonction intégrée. Vous l'invoquez dans une expression et renvoie une valeur lors de l'évaluation de l'expression. Une procédure stockée est appelée à l'aide de l'instruction **CALL**. Une procédure n'a pas de valeur de retour mais peut modifier

ses paramètres pour un contrôle ultérieur par l'appelant. Il peut également générer des ensembles de résultats à renvoyer au programme client.

b) **déclencheurs.** Un déclencheur est un objet de base de données nommé associé à une table et activé lorsqu'un événement particulier se produit pour la table, tel qu'une insertion ou une mise à jour.

c) **événements.** Un événement est une tâche qui s'exécute conformément au planning.

Les vues sont des requêtes stockées qui, lorsqu'elles sont appelées, produisent un ensemble de résultats. **Une vue agit comme une table virtuelle.**

Ce chapitre explique comment utiliser chaque type de programme enregistré et chaque vue. Des informations supplémentaires sur la syntaxe SQL pour les instructions liées à ces objets sont disponibles aux emplacements suivants:

- ❖ Pour chaque type d'objet, il existe des instructions **CREATE**, **ALTER** et **DROP** qui contrôlent les objets existants et leur définition.
- ❖ L'instruction **CALL** est utilisée pour appeler des procédures stockées.
- ❖ Les définitions de programme stockées contiennent un corps pouvant utiliser des instructions composées, des boucles, des conditions et des variables déclarées.

Chaque programme stocké contient un corps qui consiste en une instruction SQL. Cette instruction peut être une instruction composée de plusieurs instructions séparées par des points-virgules (;).

Par exemple, la procédure stockée suivante a un corps composé d'un bloc BEGIN ... END contenant une instruction SET et d'une boucle REPEAT contenant elle-même une autre instruction SET:

```
CREATE PROCEDURE dorepeat(p1 INT)
BEGIN
  SET @x = 0;
  REPEAT SET @x = @x + 1; UNTIL @x > p1 END REPEAT;
END
```

```
mysql>
mysql> DELIMITER #
mysql> CREATE PROCEDURE dorepeat(p1 INT)
-> BEGIN
-> SET @x=0;
-> REPEAT SET @x=@x+1; UNTIL @x>p1 END REPEAT;
-> END
-> #
Query OK, 0 rows affected (0.25 sec)
```

Si vous utilisez le programme client mysql pour définir un programme stocké contenant les caractères point-virgule dans sa définition, un problème se pose. Par défaut, mysql lui-même reconnaît le point-virgule en tant que délimiteur d'instruction. Vous devez donc le redéfinir temporairement pour que mysql transmette l'intégralité de la définition du programme stocké au serveur.

Pour redéfinir le délimiteur mysql, utilisez la commande `delimiter`. L'exemple suivant montre comment procéder de la manière suivante pour la procédure `dorepeat()`. Le délimiteur est remplacé par `//` pour permettre à l'ensemble de la définition d'être transmise au serveur en tant qu'instruction unique, puis restaurée; avant d'invoquer la procédure. Cela permet à la; Le délimiteur utilisé dans le corps de la procédure à transmettre au serveur plutôt que d'être interprété par mysql lui-même.

```
mysql> delimiter //

mysql> CREATE PROCEDURE dorepeat(p1 INT)
-> BEGIN
-> SET @x = 0;
-> REPEAT SET @x = @x + 1; UNTIL @x > p1 END REPEAT;
-> END
-> //
Query OK, 0 rows affected (0.00 sec)

mysql> delimiter ;
```

Essayons maintenant de rappeler la procédure **dorepeat** :

```
mysql> CALL dorepeat(1000);
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT @x;
+-----+
| @x |
+-----+
| 1001 |
+-----+
1 row in set (0.00 sec)
mysql>
```

```
mysql> CALL dorepeat(1000);
-> END
-> #
Query OK, 0 rows affected (0.00 sec)
```

```
mysql>
mysql> SELECT @x;
-> end #
+-----+
| @x    |
+-----+
| 1001  |
+-----+
1 row in set (0.00 sec)
```

Vous pouvez redéfinir le délimiteur en une chaîne autre que //, et le délimiteur peut consister en un seul caractère ou en plusieurs caractères. Vous devez éviter l'utilisation du caractère barre oblique inversée ("\\") car il s'agit du caractère d'échappement pour MySQL.

Voici un exemple de fonction qui prend un paramètre, effectue une opération à l'aide d'une fonction SQL et renvoie le résultat. Dans ce cas, il n'est pas nécessaire d'utiliser un délimiteur car la définition de la fonction ne contient pas d'interne; délimiteurs d'instruction:



```
MySQL Command Line Client
mysql> CREATE FUNCTION hello(s char(20))
-> RETURNS CHAR(50) DETERMINISTIC
-> RETURN CONCAT('Hello', s, '!');
Query OK, 0 rows affected (0.05 sec)

mysql> SELECT HELLO('WORLD');
+-----+
| HELLO('WORLD') |
+-----+
| HelloWorld!    |
+-----+
1 row in set (0.05 sec)

mysql> SELECT HELLO('Nigam');
+-----+
| HELLO('Nigam') |
+-----+
| HelloNigam!    |
+-----+
1 row in set (0.00 sec)

mysql> _
```

Application :

```
mysql> CREATE FUNCTION Salut(s CHAR(20))
-> RETURNS CHAR(50) DETERMINISTIC
-> RETURN CONCAT('Bonjour',s,'!');
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT salut(' Le Congo');
+-----+
| salut(' Le Congo') |
+-----+
| Bonjour Le Congo!  |
+-----+
1 row in set (0.00 sec)
```

UTILISATION DE ROUTINES STOCKEES (PROCEDURES ET FONCTIONS)

Les routines stockées (procédures et fonctions) sont supportées dans **MySQL 5.1**. Une routine stockée est un ensemble d'instructions SQL pouvant être stockées sur le serveur.

Une fois que cela est fait, les clients n'ont plus besoin de réémettre les relevés individuels mais peuvent se référer à la routine stockée.

Les routines stockées nécessitent la table **proc** dans la base de données **mysql**. Cette table est créée lors de la procédure d'installation de **MySQL 5.1**. Si vous effectuez une mise à niveau vers **MySQL 5.1** à partir d'une version antérieure, veillez à mettre à jour vos tables de droits pour vous assurer que la table **proc** existe.

Les routines stockées peuvent être particulièrement utiles dans certaines situations:

- ❖ Lorsque plusieurs applications clientes sont écrites dans différentes langues ou fonctionnent sur des plates-formes différentes, mais doivent exécuter les mêmes opérations de base de données.
- ❖ **Quand la sécurité est primordiale.** Les banques, par exemple, utilisent des procédures stockées et des fonctions pour toutes les opérations courantes. Cela fournit un environnement cohérent et sécurisé et des routines peuvent garantir que chaque opération est correctement consignée. Dans une telle configuration, les applications et les utilisateurs n'auraient aucun accès direct aux tables de la base de données, mais ne pourraient exécuter que des routines stockées spécifiques.

Les routines stockées peuvent améliorer les performances car moins d'informations doivent être envoyées entre le serveur et le client. Le compromis est que cela augmente la charge sur le serveur de base de données, car une plus grande partie du travail est effectuée du côté serveur et moins du côté client (application). Considérez cela si plusieurs ordinateurs clients (tels que les serveurs Web) sont desservis par un ou plusieurs serveurs de base de données.

Les routines stockées vous permettent également d'avoir des bibliothèques de fonctions dans le serveur de base de données. Il s'agit d'une fonctionnalité partagée par les langages d'application modernes qui permettent une telle conception en interne (par exemple, à l'aide de classes). L'utilisation de ces fonctionnalités de langage d'application client est bénéfique pour le programmeur même en dehors du domaine d'utilisation de la base de données.

MySQL respecte la syntaxe **SQL: 2003** pour les routines stockées, également utilisée par **IBM DB2**.

L'implémentation **MySQL** des routines stockées est toujours en cours. Toute la syntaxe décrite ici est prise en charge et toutes les limitations et extensions sont documentées le cas échéant.

QUESTIONS D'AUTO-EVALUATION

1. Un _____ désigne en fait collectivement des fonctions stockées, des procédures autonomes, des fonctions packagées et des procédures autonomes.
2. Un _____ est un objet de base de données nommé associé à une table et activé lorsqu'un événement particulier se produit pour la table, tel qu'une insertion ou une mise à jour.
3. Pour redéfinir le délimiteur MySQL, utilisez la commande _____.

III. GESTION DES PROCEDURES STOCKEES

III.1. CREATE PROCEDURE ET CREATE FUNCTION

```
CREATE
[DEFINER = {user | CURRENT_USER }]
PROCEDURE sp_name ([proc_parameter[,...]])
[characteristic ...] routine_body
```

```
CREATE
[DEFINER = {user | CURRENT_USER }]
FUNCTION sp_name ([func_parameter[,...]])
RETURNS type
[characteristic ...] routine_body
```

```
proc_parameter:
[ IN | OUT | INOUT ] param_name type
```

```
func_parameter:
param_name type
```

```
type:
Any valid MySQL data type
```

```
characteristic:
LANGUAGE SQL
| [NOT] DETERMINISTIC
| {CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
| SQL SECURITY {DEFINER | INVOKER }
| COMMENT 'string'
```

```
routine_body:
Valid SQL procedure statement
```

L'instruction **CREATE FUNCTION** est également utilisée dans **MySQL** pour prendre en charge les fonctions **UDF** (fonctions définies par l'utilisateur). Un fichier UDF peut être considéré comme une fonction stockée externe.

Remarque: Les fonctions stockées partagent leur espace de noms avec des fonctions définies par l'utilisateur.

Lorsque la routine est appelée, un **USE nom_base** implicite est exécuté (et annulé à la fin de la routine). Cela fait que la routine a la base de données par défaut donnée pendant son exécution.

Les instructions **USE** dans les routines stockées ne sont pas autorisées.

Lorsqu'une fonction stockée a été créée, vous l'invoquez en la référant dans une expression. La fonction renvoie une valeur lors de l'évaluation de l'expression. Lorsqu'une procédure stockée a été créée, vous l'invoquez à l'aide de l'instruction **CALL**.

ALTER PROCEDURE et ALTER FUNCTION

```
ALTER {PROCEDURE | FUNCTION} sp_name [characteristic ...]
characteristic:
    { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
    | SQL SECURITY { DEFINER | INVOKER }
    | COMMENT 'string'
```

Cette instruction peut être utilisée pour modifier les caractéristiques d'une procédure ou d'une fonction stockée. Vous devez disposer du privilège **ALTER ROUTINE** pour la routine. (Ce privilège est accordé automatiquement au créateur de la routine.) Si la journalisation binaire est activée, l'instruction **ALTER FUNCTION** peut également nécessiter le privilège **SUPER**.

Syntaxe de **DROP PROCEDURE** et **DROP FUNCTION**



Il existe bien une commande **ALTER PROCEDURE**, mais elle ne permet de changer ni les paramètres, ni le corps de la procédure. Elle permet uniquement de changer certaines caractéristiques de la procédure, et ne sera pas couverte dans ce cours.

```
DROP {PROCEDURE | FUNCTION} [IF EXISTS] sp_name
```

Cette instruction est utilisée pour supprimer une procédure ou une fonction stockée. C'est-à-dire que la routine spécifiée est supprimée du serveur. Vous devez disposer du privilège **ALTER ROUTINE** pour la routine. (Ce privilège est accordé automatiquement au créateur de la routine.) La clause **IF EXISTS** est une extension MySQL. Cela évite qu'une erreur ne survienne si la procédure ou la fonction n'existe pas.

DROP FUNCTION est également utilisé pour supprimer des fonctions définies par l'utilisateur.

SHOW CREATE PROCEDURE and SHOW CREATE FUNCTION

SHOW CREATE PROCEDURE proc_name

Cette instruction est une extension de MySQL. Elle retourne la chaîne de caractère exacte qui peut être utilisée pour recréer la procédure stockée nommée.

APPLICATION:

```
mysql>
mysql> SHOW CREATE PROCEDURE dorepeat;
+-----+-----+-----+-----+-----+
| Procedure | sql_mode | Create Procedure | character_set_client | collation_connection | Database Collation |
+-----+-----+-----+-----+-----+
| dorepeat |          | CREATE DEFINER='root'@'localhost' PROCEDURE `dorepeat`(p1 INT)
BEGIN
SET @x=0;
REPEAT SET @x=@x+1; UNTIL @x>p1 END REPEAT;
END | latin1 | latin1_swedish_ci | latin1_swedish_ci |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

SHOW CREATE FUNCTION SYNTAX:

SHOW CREATE FUNCTION func_name

Cette instruction affiche les informations liées à la fonction stockée nommée.

Les deux fonctions précédentes, demandent à ce que vous ayez le privilège de la routine ou l'accès à la clause **SELECT** de la table **mysql.proc**. Au cas où vous n'avez aucun privilège en ce qui regarde la routine, le résultat qui sera affiché pour la procédure **CREATE** | la fonction **CREATE** sera donc **NULL**.

```
mysql> SHOW CREATE FUNCTION salut;
+-----+-----+-----+-----+-----+
| Function | sql_mode | Create Function | character_set_client | collation_connection | Database Collation |
+-----+-----+-----+-----+-----+
| salut |          | CREATE DEFINER='root'@'localhost' FUNCTION `salut`(s CHAR(20)) RETURNS char(50) CHARSET latin1
DETERMINISTIC
RETURN CONCAT('Bonjour',s, '!') | latin1 | latin1_swedish_ci | latin1_swedish_ci |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```


QUESTIONS D'AUTO-EVALUATION

4. L'instruction.....est utilisée pour supprimer une procédure ou une fonction stockée.

IV. SHOW PROCEDURE STATUS et SHOW FUNCTION STATUS

SHOW PROCEDURE STATUS

Après exécution de cette instruction, les champs ci-dessous seront affichés:

1. BD
2. NOM
3. TYPE
4. DEFINISSEUR
5. MODIFICATEUR
6. CREE
7. TYPE_SECURITE
8. COMMENTAIRE
9. CLIENT_CONFIGUER_CARACTERE
10. CONNEXION_COLLATION
11. COLLATION BASE DE DONNEES.

En anglais cela donnerait :

1. Db
2. Name
3. Type
4. Definer
5. Modified
6. Created
7. Security_Type
8. Comment
9. character_set_client
10. collation_connection
11. Database Collation

APPLICATION:

```
mysql> SHOW PROCEDURE STATUS;
```

Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection
h_ci	dorepeat	PROCEDURE	root@localhost	2019-08-02 02:29:29	2019-08-02 02:29:29	DEFINER		latin1	latin1_swedish_ci
loi	affiche	PROCEDURE	root@localhost	2019-07-06 05:01:07	2019-07-06 05:01:07	DEFINER		utf8	utf8_general_ci
loi	insert_pro	PROCEDURE	root@localhost	2019-07-06 05:12:21	2019-07-06 05:12:21	DEFINER		utf8	utf8_general_ci

3 rows in set (0.01 sec)

SHOW FUNCTION STATUS

Après exécution de cette instruction, les mêmes champs comme ceux du dessus seront affichés.

APPLICATION :

```
mysql> SHOW FUNCTION STATUS;
```

Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection
ensp	bonjour	FUNCTION	root@localhost	2019-08-02 02:56:29	2019-08-02 02:56:29	DEFINER		latin1	latin1_swedish_ci
ensp	Salut	FUNCTION	root@localhost	2019-08-02 02:59:26	2019-08-02 02:59:26	DEFINER		latin1	latin1_swedish_ci

2 rows in set (0.00 sec)

V. L'INSTRUCTION CALL

```
CALL sp_name([parameter[,...]])
CALL sp_name[()]
```

Instruction **CALL** invoque une procédure stockée qui a été défini précédemment avec **CREATE PROCEDURE**.

Sens des paramètres

Un paramètre peut être de trois sens différents : entrant (**IN**), sortant (**OUT**), ou les deux (**INOUT**).

- **IN** : c'est un paramètre "entrant". C'est-à-dire qu'il s'agit d'un paramètre dont la valeur est fournie à la procédure stockée. Cette valeur sera utilisée pendant la procédure (pour un calcul ou une sélection par exemple).
- **OUT** : il s'agit d'un paramètre "sortant", dont la valeur va être établie au cours de la procédure et qui pourra ensuite être utilisé en dehors de cette procédure.
- **INOUT** : un tel paramètre sera utilisé pendant la procédure, verra éventuellement sa valeur modifiée par celle-ci, et sera ensuite utilisable en dehors.

Lorsque l'on crée une procédure avec un ou plusieurs paramètres, chaque paramètre est défini par trois éléments.

- Son sens : entrant, sortant, ou les deux. Si aucun sens n'est donné, il s'agira d'un paramètre **IN** par défaut.
- Son nom : indispensable pour le désigner à l'intérieur de la procédure.
- Son type : **INT**, **VARCHAR**(10),...

CALL peut passer des valeurs à son appelant en utilisant les paramètres qui ont été déclaré comme paramètre **IN**, **OUT** ou **INOUT**. Lorsque la procédure fait un retour, le programme client peut aussi obtenir le nombre de lignes affectées pour l'instruction finale exécutée au sein de la routine : Au niveau du SQL, l'appel de la fonction **ROW_COUNT()** ; depuis C, **appel la fonction mysql_affected_row()** C API.

Depuis la version MySQL 5.1.13, les procédures stockées qui ne prennent aucun argument peuvent être invoquées sans parenthèses.

Ce qui revient à dire que **CALL p()** et **CALL p** sont équivalents.

Pour avoir une valeur en retour d'une procédure à travers le paramètre **IN**, **OUT** ou **INOUT**, passez le paramètre de l'utilisateur, et alors vérifiez la valeur de la variable après le retour de la procédure. (Si vous invoquez la procédure de l'intérieur d'une autre procédure ou fonction stockée, vous pouvez aussi passer un paramètre de routine ou une variable locale de routine comme un paramètre **IN** ou **INOUT**). Pour le paramètre **INOUT**, initialisez sa valeur avant de le passer à la procédure. La procédure ci-dessous a un paramètre **OUT** que la procédure prend pour la version courante du serveur, et une valeur **INOUT** que la procédure incrémente par un de sa valeur actuelle.

```
CREATE PROCEDURE p (OUT ver_param VARCHAR(25), INOUT incr_param
INT)
BEGIN
  # Set value of OUT parameter
  SELECT VERSION() INTO ver_param;
  # Increment value of INOUT parameter
  SET incr_param = incr_param + 1;
END;
```

```
DELIMITER ;
On remet le délimiteur par défaut
```

--

Application:

```
mysql> DELIMITER #
mysql> CREATE PROCEDURE p (OUT ver_param VARCHAR(25), INOUT incr_para INT)
  -> BEGIN
  -> SELECT VERSION() INTO ver_param;
  -> SET incr_para=incr_para+1;
  -> END
  -> #
Query OK, 0 rows affected (0.05 sec)
```

Avant de faire appel à la procédure, nous devons d'abord initialiser la variable qui sera passé comme étant le paramètre **INOUT**. Et, après avoir appelé la procédure, les valeurs des deux variables seront modifiées :

```
mysql> SET @increment = 10;
mysql> CALL p(@version, @increment);
mysql> SELECT @version, @increment;
+-----+-----+
| @version      | @increment |
+-----+-----+
| 5.1.21-beta-log | 11        |
+-----+-----+
```

Application :

```
mysql>
mysql> SET @increment=10;
Query OK, 0 rows affected (0.00 sec)

mysql> CALL p(@version,@increment);
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT @version,@increment;
+-----+-----+
| @version      | @increment |
+-----+-----+
| 5.1.53-community-log | 11        |
+-----+-----+
1 row in set (0.00 sec)
```

AUTRES EXEMPLES :

Procédure avec un seul paramètre entrant

Voici une procédure qui, selon l'*id* de l'espèce qu'on lui passe en paramètre, affiche les différentes races existant pour cette espèce.

Code : SQL

```
DELIMITER | --
Facultatif si votre délimiteur est toujours |
CREATE PROCEDURE afficher_race_selon_espece (IN p_espece_id INT) --
Définition du paramètre p_espece_id
BEGIN
    SELECT id, nom, espece_id, prix
    FROM Race
    WHERE espece_id = p_espece_id; --
Utilisation du paramètre
END |
```



Notez que, suite à la création de la procédure, j'ai remis le délimiteur par défaut ;. Ce n'est absolument pas obligatoire, vous pouvez continuer à travailler avec | si vous préférez.

Pour l'utiliser, il faut donc passer une valeur en paramètre de la procédure. Soit directement, soit par l'intermédiaire d'une variable utilisateur.

Code : SQL

```
CALL afficher_race_selon_espece(1);
SET @espece_id := 2;
CALL afficher_race_selon_espece(@espece_id);
```

id	nom	espece_id	prix
1	Berger allemand	1	485.00
2	Berger blanc suisse	1	935.00
9	Rottweiler	1	600.00

id	nom	espece_id	prix
3	Singapura	2	985.00
4	Bleu russe	2	835.00
5	Maine coon	2	735.00
7	Sphynx	2	1235.00
8	Nebelung	2	985.00

Le premier appel à la procédure affiche bien toutes les races de chiens, et le second, toutes les races de chats.

Procédure avec deux paramètres, un entrant et un sortant

Voici une procédure assez similaire à la précédente, si ce n'est qu'elle n'affiche pas les races existant pour une espèce, mais compte combien il y en a, puis stocke cette valeur dans un paramètre sortant.

Code : SQL

```
DELIMITER |
CREATE PROCEDURE compter_races_selon_espece (p_espece_id INT, OUT
p_nb_races INT)
BEGIN
    SELECT COUNT(*) INTO p_nb_races
```

```
    FROM Race
    WHERE espece_id = p_espece_id;
END |
DELIMITER ;
```

Aucun sens n'a été précisé pour *p_espece_id*, il est donc considéré comme un paramètre entrant.

SELECT COUNT(*) INTO p_nb_races. Voilà qui est nouveau ! Comme vous l'avez sans doute deviné, le mot-clé **INTO** placé après la clause **SELECT** permet d'assigner les valeurs sélectionnées par ce **SELECT** à des variables, au lieu de simplement afficher les valeurs sélectionnées.

Dans le cas présent, la valeur du **COUNT(*)** est assignée à *p_nb_races*.

Procédure avec deux paramètres, un entrant et un entrant-sortant

Nous allons créer une procédure qui va servir à calculer le prix que doit payer un client. Pour cela, deux paramètres sont nécessaires : l'animal acheté (paramètre **IN**), et le prix à payer (paramètre **INOUT**).

La raison pour laquelle le prix est un paramètre à la fois entrant et sortant est qu'on veut pouvoir, avec cette procédure, calculer simplement un prix total dans le cas où un client achèterait plusieurs animaux.

Le principe est simple : si le client n'a encore acheté aucun animal, le prix est de 0. Pour chaque animal acheté, on appelle la procédure, qui ajoute au prix total le prix de l'animal en question.

Une fois n'est pas coutume, commençons par voir les requêtes qui nous serviront à tester la procédure. Cela devrait clarifier le principe. Je vous propose d'essayer ensuite d'écrire vous-mêmes la procédure correspondante avant de regarder à quoi elle ressemble.

Code : SQL

```
SET @prix = 0;                -- On initialise @prix à 0

CALL calculer_prix (13, @prix); -- Achat de Rouquine
SELECT @prix AS prix_intermediaire;

CALL calculer_prix (24, @prix); -- Achat de Cartouche
SELECT @prix AS prix_intermediaire;

CALL calculer_prix (42, @prix); -- Achat de Bilba
SELECT @prix AS prix_intermediaire;

CALL calculer_prix (75, @prix); -- Achat de Mimi
SELECT @prix AS total;
```

On passe donc chaque animal acheté tour à tour à la procédure, qui modifie le prix en conséquence. Voici quelques indices et rappels qui devraient vous aider à écrire vous-mêmes la procédure.

Le prix n'est pas un nombre entier.

Il est possible de faire des additions directement dans un **SELECT**.

Pour déterminer le prix, il faut utiliser la fonction **COALESCE ()**.

Réponse :

Secret (cliquez pour afficher)

Code : SQL

```
DELIMITER |

CREATE PROCEDURE calculer_prix (IN p_animal_id INT, INOUT p_prix
DECIMAL(7,2))
BEGIN
    SELECT p_prix + COALESCE(Race.prix, Espece.prix) INTO p_prix
    FROM Animal
    INNER JOIN Espece ON Espece.id = Animal.espece_id
    LEFT JOIN Race ON Race.id = Animal.race_id
    WHERE Animal.id = p_animal_id;
END |

DELIMITER ;
```

Et voici ce qu'affichera le code de test :

prix_intermediaire
485.00

prix_intermediaire
685.00

prix_intermediaire
1420.00

total
1430.00

AVANTAGES, INCONVENIENTS ET USAGE DES PROCEDURES STOCKEES

AVANTAGES

Les procédures stockées permettent de **réduire les allers retours entre le client et le serveur MySQL**. En effet, si l'on englobe en une seule procédure un processus demandant l'exécution de plusieurs requêtes, le client ne communique qu'une seule fois avec le serveur (pour demander l'exécution de la procédure) pour exécuter la totalité du traitement. Cela permet donc un certain **gain en performance**.

Elles permettent également de **sécuriser** une base de données. Par exemple, il est possible de **restreindre les droits des utilisateurs** de façon à ce qu'ils puissent **uniquement exécuter des procédures**.

Finis les **DELETE** dangereux ou les **UPDATE** inconsidérés. Chaque requête exécutée par les utilisateurs est créée et contrôlée par l'administrateur de la base de données par l'intermédiaire des procédures stockées. Cela permet ensuite de **s'assurer qu'un traitement est toujours exécuté de la même manière**, quelle que soit l'application/le client qui le lance. Il arrive par exemple qu'une même base de données soit exploitée par plusieurs applications, lesquelles peuvent être écrites avec différents langages. Si on laisse chaque application avoir son propre code pour un même traitement, il est possible que des différences apparaissent (distraction, mauvaise communication, erreur ou autre). Par contre, si chaque application appelle la même procédure stockée, ce risque disparaît.

INCONVENIENTS

Les procédures stockées **ajoutent évidemment à la charge sur le serveur de données**. Plus on implémente de logique de traitement directement dans la base de données, moins le serveur est disponible pour son but premier : le stockage de données. Par ailleurs, certains traitements seront toujours plus simples et plus courts à écrire (et donc à maintenir) s'ils sont développés dans un langage informatique adapté. A fortiori lorsqu'il s'agit de traitements complexes. **La logique qu'il est possible**

D'implémenter avec MySQL permet de nombreuses choses, mais reste assez basique. Enfin, **la syntaxe des procédures stockées diffère beaucoup d'un SGBD à un autre.** Par conséquent, si l'on désire en changer, il faudra procéder à un grand nombre de corrections et d'ajustements.

QUESTIONS D'AUTO-EVALUATION

5. L'instruction..... peut passer des valeurs à son appelant en utilisant des paramètres qui sont déclarés comme étant des paramètres **OUT** ou **INOUT**.
6. Pour contenir ou gérer le paramètre....., exécutez l'instruction en priori à travers l'appel (**CALL**) qui configure les variables de l'utilisateur aux valeurs qui seront passées à la procédure.

RESUME

Les procédures stockées existent pour la performance des opérations des données intenses qui ne peuvent être effectuées en utilisant juste de simples syntaxes SQL, et elles exécutent ces opérations au sein d'une base de données.

1. **Procédure stockée et routine** : une procédure stockée fait référence collectivement à une fonction stockée, une procédure stockée, une fonction paquetée, et les procédures. Pour faire appel à une procédure, il est de connaître le nom de la procédure ainsi que les paramètres qui ont été définis dans la dite procédure. Concernant la fonction, il faut aussi connaître le type de données retournées.

2. **Maintenance des procédures stockées** : ce topique nous parle des fonctions comme les instructions **CREATE**, **ALTER** et **DROP** qui permet à l'utilisateur d'effectuer les opérations demandées sur les procédures et les fonctions.
3. **Show procedure status et show function status** : Ces instructions causent l'affichage des données concernant les procédures et les fonctions créées.
4. **L'instruction CALL** : Cette instruction invoque la procédure stockée qui a été définie précédemment à travers l'instruction **CREATE PROCEDURE**. **CALL** peut passer les valeurs à son appelant en utilisant paramètres qui sont déclarés comme **OUT** ou **INOUT**.

QUESTION SUR LES TERMINAUX

1. **Expliquer l'utilisation des procédures et routines stockées.**
2. **Donnez les syntaxes relatives aux instructions ci-dessous :**
 - a. **CREATE PROCEDURE et CREATE FUNCTION**
 - b. **ALTER PROCEDURE et ALTER FUNCTION**
 - c. **SHOW CREATE PROCEDURE et SHOW CREATE FUNCTION**
3. **Donnez la syntaxe relative à l'instruction CALL.**

FIN!